

VPLocator: Vulnerability Patch Localization Model Based on LLM

Zhiyu Zhang, Jianwei Zhuge, and Xiaokang Zhou*

Abstract: The reuse of third-party open source software (OSS) has become increasingly common, leading to the inadvertent introduction of external vulnerabilities and subsequent security issues. While security patches play a crucial role in software security, the lack of vulnerability patch datasets poses a challenge. In this paper, we introduce VPLocator, an automated approach matching vulnerabilities with commit patches based on large language model (LLM). VPLocator utilizes BERT to extract deep semantic interactive relation between vulnerability description and commit message, and integrates these features with manually selected and extracted features. By training a prediction model based on a multilayer perceptron (MLP), primary VPLocator achieves an average recall rate of 98.2%. Furthermore, advanced VPLocator employs ChatGPT to analyze difference between the code before and after commit modification. By incorporating semantic features from the descriptive understanding text analyzed by ChatGPT, VPLocator enhances precision from 47.7% to 78.7%.

Key words: security patch; vulnerability analysis; large language model (LLM); localization

1 Introduction

Edge computing is a distributed computing paradigm that brings computation and data storage closer to the location where it is needed, typically at the edge of the network. Instead of relying solely on centralized cloud servers, edge computing enables processing data closer to the source. Edge intelligence seeks to leverage intelligent capabilities within edge computing devices such as sensors, mobile devices, local servers^[1], smart home devices, and industrial automation equipment, offering a wide range of applications in Internet of Things (IoT) field^[2–4].

- Zhiyu Zhang and Jianwei Zhuge are with the Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing 100084, China. E-mail: zhiyu-zh21@mails.tsinghua.edu.cn; zhugejw@tsinghua.edu.cn.
- Jianwei Zhuge is with the Zhongguancun Laboratory, Beijing 100094, China.
- Xiaokang Zhou is with the Faculty of Data Science, Shiga University, Hikone 522-0069, Japan. E-mail: zhou@biwako.shiga-u.ac.jp.

* To whom correspondence should be addressed.

Manuscript received: 2024-07-01; revised: 2024-10-08;
accepted: 2024-10-19

As Internet and software-related technologies continue to advance, the prevalence of reusing third-party open source software (OSS) on the edge devices has escalated, especially in IoT field. While code cloning enhances program writing efficiency in projects, it can also introduce external vulnerabilities unintentionally, potentially resulting in security concerns.

Code clone refers to the presence of identical or similar code fragments in code repositories, a prevalent occurrence in software development. Code clone can potentially introduce security vulnerabilities. For example, one of common vulnerabilities & exposures (CVE), CVE-2021-44228^[5] is a remote code execution (RCE) vulnerability in Log4j and Spring. According to the report^[6], over 8% of third-party libraries in the Maven ecosystem are affected by the Log4j vulnerability. Given the extensive user base of the Maven ecosystem and the widespread adoption of this component by popular frameworks, the impact of this vulnerability is far-reaching.

In the real world, long-term exposure to N-Day vulnerabilities can continue to present persistent and significant security risks to software components.

Statistical data^[7] reveals that, on average, an application software relies on 148 dependencies, with 10% of these dependencies directly contributing vulnerabilities to the project's codebase. Among the 60,648 open source projects frequently referenced by application software, each typically depends on 5.7 dependencies, with 62% of them introducing vulnerable code, either directly or indirectly.

Detecting known vulnerabilities which are within open source software code necessitates comparing the code before and after the application of patch. Security patches play a pivotal role in open source software security, not only addressing vulnerabilities but also contributing significantly to various academic research downstream. Security patches are often used as input or research objects in tasks such as dangerous code clone detection^[8] and patch existence detection^[9, 10]. Additionally, research on the lifecycle of security patches^[11] and the dissemination of security patches through upstream and downstream software^[12] utilize security patches as research objects.

Presently, real-world vulnerability datasets of significant scale as shown in the national vulnerability database (NVD) encompass critical research data such as vulnerability names, types, severity levels, and attack vectors. Nonetheless, these repositories solely contain patch information and do not encompass the source code of vulnerable program or the patch code employed to remedy these vulnerabilities. Most open source software projects use Git for version control, which means that security patches are often present in the form of historical commits. While some vulnerabilities in the NVD have related links pointing to security patches, the majority of vulnerabilities lack patch information.

Current methods for identifying vulnerability patches predominantly hinge on regular matching techniques^[13] which exhibit a coverage rate below 60%. PatchScout^[14], proposed by Tan et al, extracts statistical information that reflects the correlation between vulnerability description and commit message. By training a RankNet model, this approach endeavors to identify a series of patches for each specific vulnerability, achieving a recall rate of 69.49% for the top commit as a right patch.

However, ranking based approaches require experts to validate the results and cannot provide definitive answers on matching. In many scenarios, such as in the case of software applications tailored for hospitals,

banks, police offices, and schools, there is often a scarcity of security experts available to validate the results of vulnerabilities patch. In this work, we introduce VPLocator, an automated approach matching vulnerabilities with commit patches based on large language model (LLM). VPLocator utilizes bidirectional encoder representation from transformers (BERT^[15]) to extract deep semantic interactive relation between vulnerability description and commit message, integrating these features with manually selected and extracted features, such as the public time interval of vulnerability and commit, commit code location information, and function names in commit code. By training a prediction model based on a multilayer perceptron (MLP), primary VPLocator achieves an average recall rate of 98.2%. Furthermore, advanced VPLocator employs chat generative pre-trained transformer (ChatGPT^[16]) to analyze difference between the code before and after commit modification. By incorporating semantic features from descriptive understanding text analyzed by ChatGPT, VPLocator enhances precision from 47.7% to 78.7%.

Contribution and innovation of the work are stated as followed:

1. An automated classification approach that automatically pairs vulnerabilities with patches, eliminating the need for manual intervention.
2. An innovative application extracting deep semantic interactive features from vulnerability description and commit message texts based on BERT^[15].
3. An innovative application generating descriptive understanding texts of main code difference before and after a commit based on ChatGPT^[16].

The remainder of this paper is organized as follows: Section 2 discusses related work in vulnerability patch localization. Section 3 details the framework of VPLocator. Section 4 reports the experimental analysis and evaluation. Finally, Section 5 provides a conclusion to this paper and discusses possible future works.

2 Related Work

Within the CVE vulnerability database, every vulnerability is allocated a distinct identifier, facilitating the discovery of associated patches through the search for the CVE-ID within the commit. Additionally, partial supplementary references referring to commits can aid in the localization of

patch^[13, 17–19].

The existing methods for locating vulnerability patch are mainly based on matching techniques. In mainstream approaches, vulnerability patches are primarily matched based on the following three aspects of information:

CVE-ID^[13]: In the NVD, each vulnerability has a unique identifier. By searching for the CVE-ID in the submitted code, it can be determined whether the submitted code is a patch.

URLs in NVD related links with patch link format^[13, 17, 18]: In the NVD, some external resources related to vulnerabilities are saved in the database as reference links. These external resources may contain the source code of the vulnerability patch.

URLs with the “Patch” tag in related links in NVD: Referenced links are categorized in the NVD, and the presence of the “Patch” tag can be used to determine if it is a patch.

A technique named PatchScout^[14], introduced by Tan et al., extracts statistical data encompassing vulnerability identifiers, locations, types, as well as statistical details from vulnerability description and commit. This statistical information signifies the relationship between commit and vulnerability description. These extracted details are utilized to generate features and train a RankNet model. The primary objective of this method is to identify a sequence of patches for each targeted vulnerability, achieving a recall rate of 69.49% for $N = 1$ (refers to the top commit being a patch) and an 85.40% recall rate for $N = 5$ (refers to one of the top five commits being a real patch). This methodology effectively tackles the issue of conventional methods failing to match specific vulnerabilities with appropriate patches, offering potential patches for all vulnerabilities. The accuracy of the outcomes can then be validated through a limited manual review process.

Furthermore, another security patch localization method, VCMATCH^[20], was introduced by Wang et al. VCMATCH incorporates 36 statistical features characterizing the relationship between the vulnerability and the patch commit, encompassing both manually selected and extracted features and semantic features. VCMATCH leverages BERT to extract semantic features from the vulnerability description and commit message. Afterward, VCMATCH trains three classification models, and the predictions from these classifiers are consolidated using a voting-based

ranking fusion technique. Subsequently, Shen et al. introduced Patchmatch^[21] to help developers to quickly locate patches based on VCMATCH.

In the real world, the validation of a ranked list of commits for vulnerabilities by security experts is a task that is often limited by the availability of experts. Besides, one vulnerability can correspond to multiple commits^[22] which means some additional commits may be required to complement or rectify an initial commit. But in practice, these additional commits are commonly not included in recent security patch localization works.

Therefore, this paper introduces VPLocator, an automated approach matching vulnerabilities with commit patches, aiming to reduce the reliance on manual intervention and locating all associated commit patches.

3 Methodology

In this section, we first introduce framework of each stage of VPLocator (as shown in Figs. 1 and 2). Then we describe the details of our proposed model.

The primary VPLocator was designed to integrate innovative deep semantic interactive features based on BERT, along with traditional manually selected and extracted features derived from vulnerability and commit data. By inputting a pair of vulnerability and commit, the primary VPLocator determines whether they are matched. While the recall rate showing significant improvement, the precision rate experienced a decline. When primary VPLocator outputs a match result, the input is then passed to the advanced VPLocator. The main modified hunk pair refers to the main code modification before and after a commit. The advanced VPLocator analyzes the main modified hunk pair based on ChatGPT to enhance precision and accuracy.

3.1 Data collection

In this section, we detail information extracted from vulnerabilities and commits we gathered (as shown in Fig. 3). We utilize web crawlers to extract vulnerability description, information and affected common platform enumeration (CPE) from NVD. Vulnerability information includes CVE-ID, severity vector, reference, id of common weakness enumeration (CWE), publish date, and last modified date.

We gather commit message, information and diff text from GitHub and GitLab repositories. Example Listing

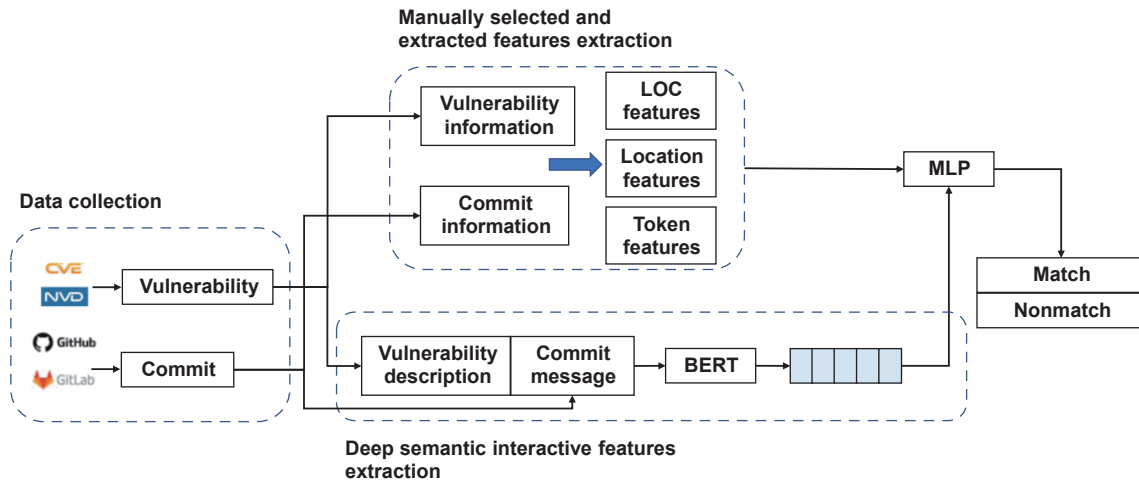


Fig. 1 Overall framework of primary VPLocator.

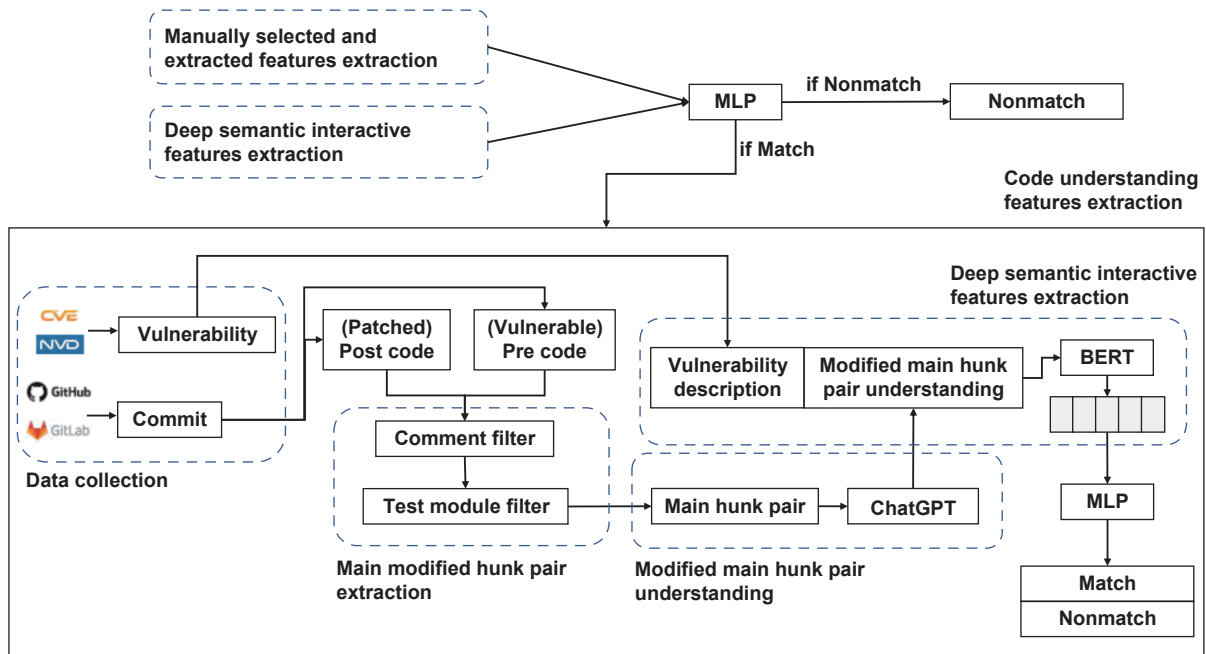


Fig. 2 Overall framework of advanced VPLocator.

1 shows extracted commit data of an OpenSSL commit patch addressing the vulnerability CVE-2022-4450.

We manually clone each git repository associated with each affected CPE and gather all commits from the repository.

Commit information, such as commit hash, author, and date, is directly extracted from git logs based on pattern matching with the corresponding keywords. Commit hash is the unique hash code of every single commit in the git repository. From the commit diff text, we extract information including pre path name, post path name, function declaration, pre lines number, post lines number, pre code, and post code. Example Listing

2 presents one hunk pair data of diff text in Listing 1.

In Git, a “hunk” refers to a portion of a diff that represents a specific set of changes made to a file. Each commit may contain multiple files diff due to modifications in multiple files, with each file diff potentially containing multiple modified code fragments referred to as “hunk pairs” in this paper. Pre path name and post path name refer to the file path of the modified file in the repository before and after the commit. Function declaration refers to the function with modified code. The pre and post line numbers consist of two integers: the first denotes the starting line of the pre or post file, while the second indicates

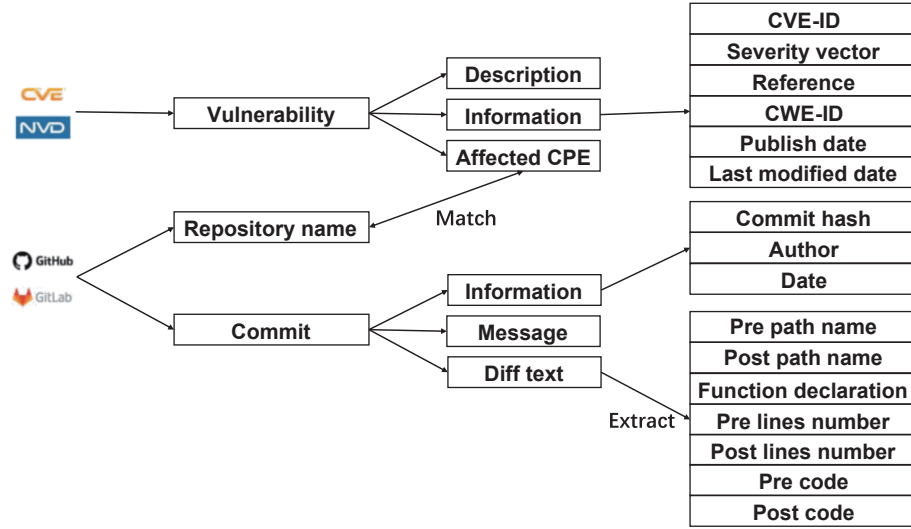


Fig. 3 Framework of VPLocator dataset.

```
{
  "CVE-ID": "CVE-2022-4450",
  "Url": "https://github.com/openssl/openssl/commit/bbcb509bd046b34c...",
  "Repository Name": "OpenSSL",
  "Commit Hash": "bbcb509bd046b34c...",
  "Author": [
    "Matt Caswell"
  ],
  "Date": "Tue Dec 13 14:54:55 2022+0000",
  "Message": "Avoid dangling ptrs in header and data params for PEM_read_bio_ex. In the event of a failure in PEM_read_bio_ex() we free the buffers we allocated for the header and data buffers. However we were not clearing the ptrs stored in *header and *data. Since, on success, the caller is responsible for freeing these ptrs this can potentially lead to a double free if the caller frees them even on failure. Thanks to Dawei Wang for reporting this issue. Based on a proposed patch by Kurt Roeckx.",
  "DiffText": "diff--gita/crypto/pem/pem_lib.cb/crypto/pem/pem_lib.c\nindex d416d939ea..328c30cddb 100644\n---a/crypto/pem/pem_lib.c\n+++b/crypto/pem/pem_lib.c\n@@ -957,7+957,9 @@ int PEM_read_bio_ex(BIO *bp, char **name_out, char **header, *data = pem_malloc(len, flags);\nif (*header == NULL||*data == NULL){\n    pem_free(*header, flags, 0);\n    pem_free(*data, flags, 0);\n    goto end;\n}\nBIO_read(headerB, *header, headerlen);",
  "Pre Path Name": "crypto/pem/pem_lib.c",
  "Post Path Name": "crypto/pem/pem_lib.c",
  "Function Declaration": "int PEM_read_bio_ex(BIO *bp, char **name_out, char **header, *data = pem_malloc(len, flags);\nif (*header == NULL||*data == NULL){\n    pem_free(*header, flags, 0);\n    pem_free(*data, flags, 0);\n    *data = NULL;\n    goto end;\n}\nBIO_read(headerB, *header, headerlen);",
  "Pre Lines Number": [957, 7],
  "Post Lines Number": [957, 9],
  "Pre Code": "data = pem_malloc(len, flags);\nif (*header == NULL||*data == NULL){\n    pem_free(*header, flags, 0);\n    pem_free(*data, flags, 0);\n    goto end;\n}\nBIO_read(headerB, *header, headerlen);",
  "Post Code": "data = pem_malloc(len, flags);\nif (*header == NULL||*data == NULL){\n    pem_free(*header, flags, 0);\n    pem_free(*data, flags, 0);\n    *data = NULL;\n    goto end;\n}\nBIO_read(headerB, *header, headerlen);",
}
```

Listing 1 Commit data.

the number of affected lines. Pre code refers to the lines in the diff text with prefix “-”, along with other shared code that have no prefix, while post code refers to the lines in the diff text that have no prefix and with

```
{
  "Repository Name": "OpenSSL",
  "Commit Hash": "bbcb509bd046b34c...",
  "Pre Path Name": "crypto/pem/pem_lib.c",
  "Post Path Name": "crypto/pem/pem_lib.c",
  "Function Declaration": "int PEM_read_bio_ex(BIO *bp, char **name_out, char **header, *data = pem_malloc(len, flags);\nif (*header == NULL||*data == NULL){\n    pem_free(*header, flags, 0);\n    pem_free(*data, flags, 0);\n    goto end;\n}\nBIO_read(headerB, *header, headerlen);",
  "Pre Lines Number": [957, 7],
  "Post Lines Number": [957, 9],
  "Pre Code": "data = pem_malloc(len, flags);\nif (*header == NULL||*data == NULL){\n    pem_free(*header, flags, 0);\n    pem_free(*data, flags, 0);\n    goto end;\n}\nBIO_read(headerB, *header, headerlen);",
  "Post Code": "data = pem_malloc(len, flags);\nif (*header == NULL||*data == NULL){\n    pem_free(*header, flags, 0);\n    pem_free(*data, flags, 0);\n    *data = NULL;\n    goto end;\n}\nBIO_read(headerB, *header, headerlen);",
}
```

Listing 2 Hunk pair data of diff text.

the prefix “+”. Affected CPE in format “cpe :part: vendor:product:version:update:edition:language” indicates which product is affected by the vulnerability.

To correctly label the vulnerability and patch, we match CVE-IDs in commits message with regular expression, verify additional references from vulnerability information, and manually identify certain patches based on human expertise.

3.2 Primary VPLocator

Primary VPLocator (see framework in Fig. 1) aims to

enhance the patch coverage for vulnerabilities and consists of two main components. The first part involves extracting manually selected and extracted features like the time interval between commit submission and vulnerability publication or modification, as well as shared non-stop words in vulnerability description and commit message. The second part predominantly utilizes the BERT model to extract semantic interactive features between the vulnerability description and commit message. Subsequently, the features from both parts are fed into an MLP classification model, which outputs either a Match or Nonmatch classification result.

3.2.1 Manually selected and extracted features

In this section, we introduce the manually selected and extracted features. Table 1 presents the specifics of these features. In addition to those found in the previous work VCMATCH^[20], we extract additional features (highlighted in bold in the table).

LOC Features Vulnerability patches typically involve modifications to fewer lines of code compared to commits implementing functional requirements^[23]. Therefore, we compute the number of lines of code (LOC) altered by a commit, encompassing the total LOC added, deleted, and modified. Nonetheless, there are numerous instances where a single patch commit

Table 1 Features that are used by VPLocator^[20].

Dimension	Feature	Description
LOC feature	Main Code¹ Added Num	# of lines of main code added in the commit.
	Main Code Deleted Num	# of lines of main code deleted in the commit.
	Main Code Modified Num	# of lines of main code modified in the commit.
	Code Added Num	# of lines of code added in the commit.
	Code Deleted Num	# of lines of code deleted in the commit.
	Code Modified Num	# of lines of code modified in the commit.
Location feature	Author ID	# of index number of commit authors list
	Modified Time Interval	Time interval between commit date and NVD last modified date.
	Published Time Interval	Time interval between commit date and NVD published date.
	Same Path Name Num	# of file paths that exist in both commit and vul ² des ³ .
	Same Path Name Ratio	# of same file paths / # of file paths modified by the commit.
	Unrelated Path Name Num	# of file paths that exist in commit but not marked in the vul des.
	Same File Name Num	# of files that exist in both commit and vul des.
	Same File Name Ratio	# of same files / # of files modified by the commit.
	Unrelated File Name Num	# of files that exist in commit but not marked in the vul des.
	Same Function Name Num	# of functions that exist in both commit and vul des.
	Same Function Name Ratio	# of same functions / # of functions modified by the commit.
	Unrelated Function Name Num	# of functions that exist in commit but not marked in the vul des.
Token feature	Shared-Vul-Msg-Word Num	# of shared non-stop words between vul des and commit message.
	Shared-Vul-Msg-Word Ratio	# of Shared-Vul-Msg-Words / # of words in vul description.
	Max of Shared-Vul-Msg-Word	The max of the frequencies for all Shared-Vul-Msg-Words.
	Sum of Shared-Vul-Msg-Word	The sum of the frequencies for all Shared-Vul-Msg-Words.
	Average of Shared-Vul-Msg-Word	The average of the frequencies for all Shared-Vul-Msg-Words.
	Variance of Shared-Vul-Msg-Word	The variance of the frequencies for all Shared-Vul-Msg-Words.
	Shared-Vul-Code-Word Num	# of shared non-stop words between vul des and code diff text.
	Shared-Vul-Code-Word Ratio	# of Shared-Vul-Code-Words / # of words in vul description.
	Max of Shared-Vul-Code-Word	The max of the frequencies for all Shared-Vul-Code-Words.
	Sum of Shared-Vul-Code-Word	The sum of the frequencies for all Shared-Vul-Code-Words.
	Average of Shared-Vul-Code-Word	The average of the frequencies for all Shared-Vul-Code-Words.
	Variance of Shared-Vul-Code-Word	The variance of the frequencies for all Shared-Vul-Code-Words.

¹ “main code”: pertinent code segments, excluding comments and test module code.

² “vul”: an abbreviation for “vulnerability”.

³ “des”: an abbreviation for “description”.

encompasses both vulnerability modifications and other alterations. The presence of lines modified for other purposes can potentially introduce confusion to the model. Therefore, we designate “main code” as the pertinent code, excluding comments and test module code. Subsequently, we extract the number of lines added, deleted, and modified in the main code modification segments.

Location Features and **Token Features** are mainly same as the previous work VCMATCH^[20]. Statistical numbers and ratios of same and unrelated path names, file names, and function names, along with the maximum, sum, average, and variance of shared words in vulnerability description and commit message and commit diff text, serve as indicators of the potential relationship between vulnerabilities and commits. Moreover, security experts are more inclined to submit commit patches compared to other engineers. Therefore, we consider the commit author and assign an index to it. A recent patch, as opposed to an older one, is more likely to have a commit date closer to the NVD last modified date. So we extract the modified time interval additionally.

3.2.2 Deep semantic interactive features

The deep semantic interactive features reveal the correlation between the vulnerability description and commit message. By concatenating and encoding the vulnerability description and commit message using BERT, followed by MLP classification training with matching labels, we can discern the relationship. Unlike VCMATCH^[20], which encodes the two descriptions separately with BERT, VPLocator excels in extracting the interaction information between the descriptions more effectively.

In the BERT model^[15], the second pretraining task is “Next Sentence Prediction” which aims to extract interactive features from two sentences to predict whether the first sentence precedes the second. This capability allows it to be utilized in patch location tasks by concatenating the vulnerability description and commit message.

3.3 Advanced VPLocator

False positive results primarily stem from the fact that in real Git repositories, there are numerous commits that modify code within the same file, and sometimes even the same functions. In order to enhance precision in practical applications, we introduce the Advanced VPLocator approach (refer to the framework in Fig. 2),

which focuses on precisely matching vulnerabilities with commits by understanding the main modified hunk pairs based on ChatGPT^[16]. The vulnerability description and the understanding of hunk pairs generated by ChatGPT are combined and encoded using BERT, following a similar process as in the Primary VPLocator model, to produce either a Match or Nonmatch output. Given the high computational demands of ChatGPT, comprehending all commit code is impractical. Therefore, only entries with a Match output from the Primary VPLocator model are considered for entry into the code understanding module in Advanced VPLocator, which enhances precision. When a commit contains multiple modified hunk pairs, we combine the understanding texts of these hunk pairs into a single lengthy text.

3.3.1 Modified hunk pair understanding

Due to the lack of standardization and the incompleteness of commits message^[24], it is crucial to focus on the modified code. Leveraging a ChatGPT-powered programming tool can provide concise and accurate explanations for code snippets^[25].

Listing 3 shows an example of ChatGPT understanding for a hunk pair from Listing 2. In response to the absence of current datasets and evaluations for understanding modified hunk pairs, we have developed a manual estimation standard, detailed in Table 2.

Four standard dimensions assess hunk pair comprehension. The basic metric, Words Count, refers to the word count in ChatGPT responses, limited to a maximum length of 128(=MAXLENGTH) words in our study. In the true positive dimension, the focus is on the accurate word count and ratio. The Correct Words Count refers to the number of words that accurately describe the content, while the ratio is calculated as Correct Words Count divided by Total Words Count. The false positive dimension evaluates the count and ratio of imaginary and irrelevant words. The Fictitious Words Count and Irrelevant Words Count respectively refer to the number of words that inaccurately or irrelevantly describe the content, while the ratio is calculated as Fictitious Words Count or Irrelevant Words Count divided by Total Words Count. In addition, ChatGPT may also miss out on providing specific variables and functions for modified hunk pairs or respond correctly without addressing the key modifications. Since there are no statistical values available for the missing dimension, we assigned

Table 2 Standard of evaluation on hunk pairs understanding by ChatGPT.

Dimension	Metric	Description	Type	Range
Basic	Words Count	# of words in response.	Int	[0, MAXLENGTH]
True positive	Correct Words Count	# of words of correct understanding description.	Int	[0, Words Count]
	Correct Words Ratio	Correct Words Count / Words Count.	Double	[0, 1]
False positive	Fictitious Words Count	# of words of fictitious understanding description.	Int	[0, Words Count]
	Fictitious Words Ratio	Fictitious Words Count / Words Count.	Double	[0, 1]
	Irrelevant Words Count	# of words of irrelevant understanding description.	Int	[0, Words Count]
	Irrelevant Words Ratio	Irrelevant Words Count / Words Count.	Double	[0, 1]
Missing	Specificity	Subjective grade for specific variable or function	Int	[0, 10]
	Importance	Subjective grade for key modification description	Int	[0, 10]

ChatGPT
Version: gpt-3.5-turbo
Prompt: You are a code reviewer in English. There are code snippets before and after a commit for you to review. Please summarize the modifications concisely in under 128 words.
Question: The source code before and after a commit is provided below.

```
Before the commit:
*data = pem_malloc(len, flags);
if(*header == NULL || *data == NULL){
    pem_free(*header, flags, 0);
    pem_free(*data, flags, 0);
    goto end;
}
BIO_read(headerB, *header, headerlen);

After the commit:
*data = pem_malloc(len, flags);
if(*header == NULL || *data == NULL){
    pem_free(*header, flags, 0);
    *header = NULL;
    pem_free(*data, flags, 0);
    *data = NULL;
    goto end;
}
BIO_read(headerB, *header, headerlen);
```

What are the reasons for and methods of modification in the code?

Answer: The code was modified to prevent potential double free issues. After the commit, setting *header and *data to NULL after freeing them avoids double deallocation and ensures proper memory management.

Listing 3 Hunk pair understanding by ChatGPT.

subjective grades to each entry. Specificity assesses the presence of specific variables or functions in the understanding, while importance evaluates the existence of key modifications within the commit.

3.3.2 Main modified hunk pair

During manual estimation using diff text and hunk pairs, we noticed that lower grades often arise when the patch commit code is lengthy and includes additional modifications like adding comments or test modules. Therefore, we have devised two filters to extract the

main modified hunk pair, focusing on relevant code segments to reduce the impact of unrelated changes. The comment filter screens out hunk pairs that are comments and the test module filter excludes modifications in test modules.

4 Evaluation

In this section, we focus on VPLocator Dataset collecting approach, and define symbol $P(v, c)$, *Label*, *Model*. Subsequently, we present the results of three experiments, effectiveness of deep semantic interactive features, evaluation on modified code understanding, and results of VPLocator in vulnerability patch localization. Finally, we discuss the advantages and limitations of VPLocator.

4.1 Dataset

We extract information of vulnerabilities and commits (as shown in Fig. 3), forming the “VPLocator Dataset” which is then stored in a MongoDB database. We utilize web crawlers to extract vulnerability description, information (including CVE-ID, severity vector, references, CWE-ID, publish date, and last modified date) and affected CPEs from NVD.

In Git, a “hunk” refers to a portion of a diff that represents a specific set of changes made to a file. Each commit may contain multiple file diffs due to modification in multiple files, with each diff potentially containing multiple modified code fragments referred to as “hunk pairs” in this paper. Pre path name and post path name refer to the file path of the modified file in the repository before and after the commit, extracted by matching the pattern “diff -git a/(Pre path name) b/(Post path name)”. Function declaration, pre and post lines number are matched with pattern “@@ -(Pre lines number) +(Post lines number) @@ (Function

declaration)". The pre and post line numbers consist of two integers: the first denotes the starting line of the pre or post file, while the second indicates the number of affected lines. In cases where there are multiple lines in the function declaration due to numerous parameters, the first line containing the essential function name is selected. Pre code refers to the lines in the diff text that have the prefix "+" removed, while post code refers to the lines in the diff text where lines with the prefix "-" are removed. Affected CPEs are in the similar format of "cpe:2.3:a:openssl:openssl:...". We manually clone each git repository associated with each affected CPE and gather commits from the repository.

To correctly label the vulnerabilities and patches, we match CVE-IDs, verify additional references, and manually identify certain patches. Certain vulnerability-fixing commits explicitly contain the relevant CVE-ID, as seen in the OpenSSL commit addressing CVE-2022-4450 (refer to the commit message in Listing 1). To identify these CVE-IDs within commits message, we employ a regular expression "CVE-[0-9]4-[0-9]1,8". Additionally, references from vulnerability information often include URLs for patches. We utilize regular expressions such as "http[s]?://github.com/[a-zA-Z0-9]+/[a-zA-Z0-9]+/commit/[a-f0-9]+" and a similar pattern for GitLab. Finally, we find certain patches through human expertise, such as using search engines to look for keywords, reading vulnerability reports, and searching for commits with explicit file name and variable name.

354 git repositories were selected based on their relevance to email, routers, domain name system (DNS), content delivery network (CDN), public key infrastructure (PKI) and transport layer security (TLS). The VPLocator Dataset identifies patch commits by checking extra references and matching CVE-IDs. This dataset comprises 2755 CVE vulnerabilities from NVD and find 2456 patch commits from Git repositories for 1071 CVE vulnerabilities. We randomly choose 10 commits which aren't patches. In amount, 32417 commits from 354 git repositories are stored. And there are 267 259 modified hunk pairs extracted from commits in VPLocator Dataset.

For comparative analysis, we utilize the VCMATCH^[20] dataset, which includes 1669 vulnerabilities along with 1855 associated patch commits from 10 repositories. Additionally, a random selection of 2000 commits is included as negative samples for each vulnerability, so

VCMATCH has 3 711 855 entries.

4.2 Basic definition

We define symbol $P(v, c)$, Label, Model for better describing evaluation indicators.

Definition 1 $P(v, c)$ denotes the vulnerability-commit pair, where P stands for "pair", v for "vulnerability" and c for "commit".

Definition 2 $\text{Label}(P(v, c))$ refers to the actual match status, indicating whether the commit c is the patch of vulnerability v .

$$\text{Label}(P(v, c)) = \begin{cases} 1, & c \text{ is the patch of } v \\ 0, & c \text{ isn't the patch of } v \end{cases}$$

Definition 3 $\text{Model}(P(v, c))$ refers to the predicted match status, indicating whether the model outputs Match or NonMatch when provided with $P(v, c)$.

$$\text{Model}(P(v, c)) = \begin{cases} 1, & \text{the model outputs Match} \\ 0, & \text{the model outputs Nonmatch} \end{cases}$$

Input $P(v, c)$ to Model

4.3 Implementation detail of deep semantic interactive features

In this section, we provide a detailed overview of the implementation of BERT in our approach. We discuss the specific tools, techniques and parameters used in the implementation process. This includes a breakdown of the algorithms employed, the software frameworks utilized. By delving into these implementation details, we aim to provide a comprehensive understanding of the practical aspects of our work.

We conducted experiments using the BERT model from the transformer module in the PyTorch framework, with the optimizer set as AdamW. The configure parameters of BERT model we use is seen in Table 3.

4.4 Effectiveness of deep semantic interactive features

We design an experiment with three models for classifying whether a vulnerability is related to a commit. (as shown in Fig. 4). The TFIDF-COS model employs the term frequency-inverse document frequency (TFIDF^[26]) algorithm to compute the TFIDF vectors of vulnerability description and commit message. Subsequently, it calculates the cosine similarity^[27] between the vectors, providing a normalized measure of the similarity between the two texts. Then select a threshold; if the similarity is higher than it, output Match, otherwise output Nonmatch. The

Table 3 Configure parameters of BERT model.

Parameter name	Value
Attention probs dropout prob	0.1
Hidden dropout prob	0.1
Hidden size	768
Initializer range	0.02
Intermediate size	3072
Max position embeddings	512
Num attention heads	12
Num hidden layers	12
Pad token id	0
Type vocab size	2
Vocab size	30 522
Hidden act	gelu
Position embedding type	absolute
Gradient checkpointing	false
Use cache	true

Separate-BERT model takes the vulnerability description and commit message as input, embedding them into two separate vectors using BERT. These vectors are then fed into an MLP classification model, which outputs either Match or Nonmatch. In contrast, the Interactive-BERT model concatenates the vulnerability description and commit message before following the same process.

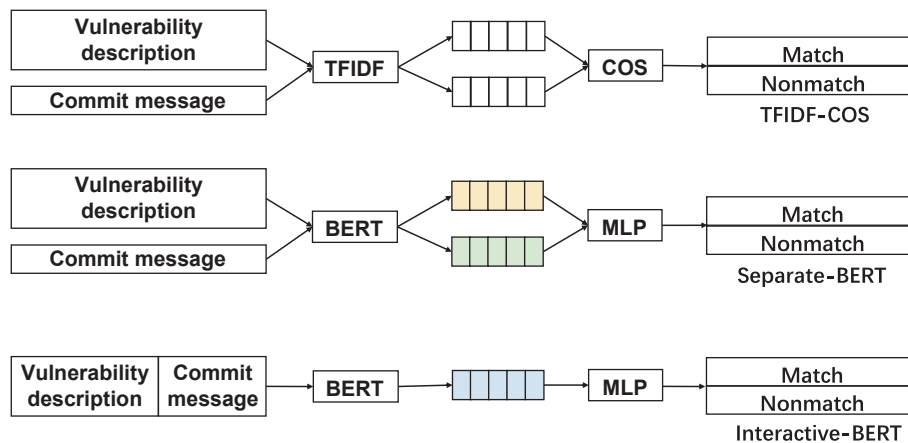
Using the VPLocator Dataset with a positive-to-negative sample ratio of 1:10 for training and testing (with a training-to-testing ratio of 9:1), the results are

displayed in Table 4. The Interactive-BERT model exhibits significantly superior results compared to the other two models. In the BERT model^[15], the second pretraining task aims to extract interactive features from two sentences to predict whether the first sentence precedes the second. BERT can extract deep semantic interactive features by concatenating the vulnerability description and commit message. This capability allows Interactive-BERT model acting better than Separate-BERT and TFIDF-COS.

4.5 Evaluation on modified code understanding

By inputting diff text and hunk pair into ChatGPT, we can obtain explanation for the modified code. The specific prompt used is illustrated in Fig. 5. Experimenting with various prompts, we discovered that the format displayed in Fig. 5 produced the most favorable outcomes.

Each commit corresponds to a single diff text, containing various modified file diffs. Each file diff within the diff text starts with "diff -git", and modified diff begins with "----" and "+++", and may comprise multiple modified hunks. In Git, a "hunk" denotes a block of code change in an adjacent section of a file. For instance, when inspecting modification using "git diff" command, a segment delineated by two "@@" symbols signifies a hunk. Lines commencing with "-" denote deleted lines, while lines starting with "+" indicate added lines. We extracted hunk pairs by

**Fig. 4 Framework of interactive features experiment.****Table 4 Results of three models on VPLocator Dataset.**

Model name	Accuracy	Precision	Recall	F1
TFIDF-COS	0.840	0.767	0.545	0.637
Separate-BERT	0.796	0.655	0.433	0.521
Interactive-BERT	0.872	0.850	0.842	0.846

```

payload = json.dumps({
    "model": "gpt-3.5-turbo",
    "messages": [
        {
            "role": "system",
            "content": "You are a code reviewer in %s. There are code snippets before and after a commit for you to review. Please summarize the modifications concisely in under %d words." % (language, maxlength)
        },
        {
            "role": "user",
            "content": "The source code before and after a commit is provided below.\n\nBefore the commit:%s\nAfter the commit:%s\n\nQuestion: What are the reasons for and methods of modification in the code?" % (precodes, postcodes)
        }
    ]
})

```

Fig. 5 Prompt of modified hunk pair understanding.

filtering the hunks diff, generating pre code and post code. However, some hunk pairs were unrelated to vulnerabilities, such as comments and test module code. Therefore, we filtered them out, resulting in the main modified hunk pairs.

The comment filter screens out hunk pairs that are comments by identifying “//”, “/*”, “*/” in C, C++, Java, and JavaScript, “#” in Python, as well as in other programming languages. The test module filter excludes modification in test modules by identifying occurrences of keyword “test” in file name, module name, and function name.

The assessment of understanding for modified diff text, hunk pair, and main hunk pair is presented in Table 5. We randomly gathered 20 patched diffs from VPLocator Dataset and 20 other diffs that do not pertain to vulnerability patching, extracted 89 patched hunk pairs and 168 other hunk pairs, filtered out comments and test modules to focus on 65 patched main hunk pairs and 129 other hunk pairs. Subsequently, we utilized this data to prompt ChatGPT and evaluated the response text metrics using the criteria outlined in Table 2. We computed the averages for each metric, with main hunk pair scoring highest in

Correct Words Ratio, Specificity, and Importance, while receiving the lowest grades in Fictitious Words Ratio and Irrelevant Words Ratio. Main hunk pair has fewer disturbances and contain more effective code, which results in better performance compared to analyzing the entire diff text and unfiltered hunk pair in modified code understanding by ChatGPT.

The diff text contains redundant information, leading to confusion for ChatGPT. The use of fictitious words is high, while specificity and importance are low. While hunk pairs are generally understandable, the focus on vulnerability-related code modifications is lacking. The main hunk pair exhibits the highest ratio of correct words, specificity, and importance, and the lowest ratio of fictitious and irrelevant words.

4.6 Results of VPLocator in vulnerability patch localization

Using a 10-fold cross-validation approach, the result of experiment of Primary and Advanced VPLocator on VPLocator Dataset is as followed in Table 6. In comparison to the real-world scenario, the precision reaches a virtual high. While the VPLocator Dataset maintains a positive-to-negative sample ratio of 1:10, in actual Git repositories, the number of non-patch commits can range from 1000 to 1 000 000, increasing false positive rate. Therefore, the VPLocator model may perform less effectively on precision in real-world applications.

In the automated vulnerability patch matching scenario, the dataset has an imbalance in the ratio of positive to negative samples, making accuracy less meaningful. Precision measures the ratio of actual matched patches in the model's output marked as Match. Higher precision indicates fewer incorrect

Table 5 Evaluation on hunk pairs understanding by ChatGPT

Metric	Diff text		Hunk pair		Main Hunk Pair	
	Patched	Other	Patched	Other	Patched	Other
Quantity	20	20	89	168	65	129
Words count	89.6	107.2	57.8	61.1	54.0	62.3
Correct words count	48.6	51.9	45.7	47.4	47.2	53.2
Correct words ratio	0.511	0.439	0.784	0.772	0.858	0.840
Fictitious words count	27.4	33.9	6.5	7.3	4.2	6.0
Fictitious words ratio	0.301	0.324	0.117	0.121	0.088	0.096
Irrelevant words count	13.6	21.4	5.6	6.4	2.6	3.1
Irrelevant words ratio	0.188	0.237	0.099	0.107	0.054	0.064
Specificity	5.43	5.29	6.87	6.94	7.59	7.75
Importance	3.54	4.39	5.93	5.87	6.01	5.95

Table 6 Results of vulnerability patch matching models on VPLocator dataset.

Model	Accuracy	Precision	Recall	F1
Primary VPLocator	0.988	0.869	0.989	0.925
Advanced VPLocator	0.993	0.950	0.981	0.965

matches. Recall measures the coverage of vulnerabilities, which is crucial for accurately localizing vulnerability patches. Higher recall indicates more correct vulnerability patches chosen out. VCMATCH Dataset is evenly divided into 10 parts. To analyze the features, 9 parts of training data are randomly undersampled with a positive-to-negative sample ratio of 1:10, while 1 part of testing data has a positive-to-negative sample ratio of 1:2000 same as original VCMATCH Dataset. Using a 10-fold cross-validation approach, the results of the experiments on Primary and Advanced VPLocator on the VCMATCH Dataset are presented in Table 7, compared with the results from VCMATCH paper^[20]. VCMATCH is based on a ranking approach, so we select $N=1$ and $N=2$, where $N=1$ indicates considering the top commit as the patch for the input vulnerability, and $N=2$ considers the top two commits.

Recall refers to ratio of $P(v,c)$ entries where $\text{Label}(P(v,c))=1$ and $\text{Model}(P(v,c))=1$ among all $P(v,c)$ entries where $\text{Label}(P(v,c))=1$. The results indicate that the recalls of Primary VPLocator at 98.2% and Advanced VPLocator at 96.8% are higher than VCMATCH ($N=1$) at 79.9% and VCMATCH ($N=2$) at 82.8%. suggesting that VPLocator identifies more patches for vulnerabilities. However, the large quantity of negative samples leads to VPLocator having a lower precision. Precision refers to ratio of $P(v,c)$ entries where $\text{Label}(P(v,c))=1$ and $\text{Model}(P(v,c))=1$ among all $P(v,c)$ entries where $\text{Model}(P(v,c))=1$. Results show Advanced VPLocator significantly improving precision from 86.9% to 95.0% on the VPLocator Dataset (as shown in Table 6) and from 47.7% to 78.7% on the VCMATCH Dataset. Advanced VPLocator exhibits only a slight decrease in recall compared to

Primary VPLocator, achieving the highest F1 score of 0.868 compared to recent related research studies.

Deep semantic interactive features contribute to the recall on the VCMATCH Dataset, while understanding main hunk pairs enhances precision by filtering out incorrect cases where the commit message lacks precision and completeness. However, there is still a gap in precision compared to VCMATCH ($N=1$), indicating that there is room for improvement in our main hunk pairs understanding.

5 Conclusion and Future Work

We propose VPLocator, an automated approach matching vulnerabilities with commit patches based on large language models (LLM). Primary VPLocator utilizes BERT^[15] to extract deep semantic interactive relation between vulnerability description and commit message. The experimental results demonstrate that the Interactive-BERT model outperforms the TFIDF-COS model and Separate-BERT model, showcasing superior performance among the three models.

Integrating these semantic features with handcrafted features, by training a prediction model based on a MLP, Primary VPLocator achieves an average precision rate of 86.9% and recall rate of 98.9% on VPLocator Dataset (positive-to-negative sample ratio of 1:10) and precision rate of 47.7% and recall rate of 98.2% on VCMATCH Dataset (positive-to-negative sample ratio of 1:2000).

Advanced VPLocator employs ChatGPT^[16] to explain main hunk pairs which refer to pertinent code segments before and after modifications. Furthermore, we developed a manual estimation standard to assess the understanding of text in modified commits generated by ChatGPT. We evaluated the responses with modified diff text, hunk pair, and main hunk pair. The findings indicate that main hunk pair yields the most accurate understanding text. However, there is still room for improvement in the current main hunk pair. A more sophisticated filtering approach could focus on the modified code related to vulnerabilities, leading to a more thorough evaluation in conjunction with the insights provided by ChatGPT.

Through the integration of semantic features extracted from the descriptive understanding text of main modified hunk pairs analyzed by ChatGPT. Advanced VPLocator significantly boosts precision from 86.9% to 95.0% on the VPLocator Dataset and precision from 47.7% to 78.7% on the VCMATCH

Table 7 Results of vulnerability patch matching models on VCMATCH dataset.

Model	Accuracy	Precision	Recall	F1
VCMATCH ($N=1$)	0.999	0.889	0.799	0.842
VCMATCH ($N=2$)	0.999	0.460	0.828	0.591
Primary VPLocator	0.998	0.477	0.982	0.642
Advanced VPLocator	0.999	0.787	0.968	0.868

Dataset, achieving the highest F1 score 0.868 compared to recent related research studies.

However, there remains a precision gap compared to VCMatch ($N=1$), suggesting that there is scope for improvement in our approach. One potential solution could involve transitioning from a simple MLP classification model to a more advanced machine learning or deep learning model. Another approach could involve refining the extraction of main hunk pairs using a more precise algorithm that fully focuses on the contrasts in vulnerability patching code.

Additionally, we will conduct ablation experiments to assess the contribution of each class of features. Moreover, there are numerous CVEs for which patches cannot be found even through manual efforts. These types of vulnerabilities are not included in the testing dataset recently. We plan to utilize our model to identify patches for these vulnerabilities to evaluate its effectiveness in real-world scenarios.

Acknowledgment

This work was supported by the National Key Research and Development Program “Strategic Technology Innovation Cooperation” Key Special Project (No. SQ2024YFE0201727).

References

- [1] Y. Chen, F. Zhao, Y. Lu and X. Chen, Dynamic Task Offloading for Mobile Edge Computing with Hybrid Energy Supply, *Tsinghua Science and Technology*, vol. 28, no. 3, pp. 421–432, 2023.
- [2] Z. Lian and F. Tian, DeepSI: A Sensitive-Driven Testing Samples Generation Method of Whitebox CNN Model for Edge Computing, *Tsinghua Science and Technology*, vol. 29, no. 3, pp. 784–794, 2024.
- [3] Y. Qu, L. Ma, W. Ye, X. Zhai, S. Yu, Y. Li, and D. Smith, Towards Privacy-Aware and Trustworthy Data Sharing Using Blockchain for Edge Intelligence, *Big Data Mining and Analytics*, vol. 6, no. 4, pp. 443–464, 2023.
- [4] X. Sun, Y. He, D. Wu and J. Z. Huang, Survey of Distributed Computing Frameworks for Supporting Big Data Analysis, *Big Data Mining and Analytics*, vol. 6, no. 2, pp. 154–169, 2023.
- [5] U. Changsan and P. Boonyopakorn, Log4shell Investigate Based On Generic Computer Forensic Investigation Model, in *Proc. 20th Int. Conf. on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology*, Nakhon Phanom, Thailand, 2023, pp. 1–6.
- [6] Y. Wu, Z. Yu, M. Wen, Q. Li, D. Zou and H. Jin, Understanding the Threats of Upstream Vulnerabilities to Downstream Projects in the Maven Ecosystem, in *Proc. IEEE/ACM 45th Int. Conf. on Software Engineering*, Melbourne, Australia, 2023, pp. 1046–1058.
- [7] Synopsys, 8th annual state of the software supply chain, <https://www.sonatype.com/resources/state-of-the-software-supply-chain-2022/introduction>, 2022.
- [8] J. Jang, A. Agrawal and D. Brumley, ReDeBug: Finding Unpatched Code Clones in Entire OS Distributions, in *Proc. IEEE Symposium on Security and Privacy*, San Francisco, CA, USA, 2012, pp. 48–62.
- [9] S. Kim, S. Woo, H. Lee and H. Oh, VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery, in *Proc. IEEE Symposium on Security and Privacy*, San Jose, CA, USA, 2017, pp. 595–614.
- [10] Z. Li, D. Zou, S. Xu, H. Jin, H. Qi, and J. Hu, VulPecker: an automated vulnerability detection system based on code similarity analysis. in *Proc. 32nd Annual Conf. on Computer Security Applications*, New York, NY, USA, 2016, pp. 201–213.
- [11] G. V. Marconato, V. Nicomette and M. Ka n che, Security-related vulnerability life cycle analysis, in *Proc. 7th Int. Conf. on Risks and Security of Internet and Systems*, Cork, Ireland, 2012, pp. 1–8.
- [12] R. J. Ellison and C. Woody, Supply-Chain Risk Management: Incorporating Security into Software Development, in *Proc. 43rd Hawaii Int. Conf. on System Sciences*, Honolulu, HI, USA, 2010, pp. 1–10.
- [13] H. Perl, S. Dechand, M. Smith, D. Arp, F. Yamaguchi, K. Rieck, S. Fahl, and Y. Acar, VCCFinder: Finding Potential Vulnerabilities in Open-Source Projects to Assist Code Audits, in *Proc. 22nd ACM SIGSAC Conf. on Computer and Communications Security*, New York, NY, USA, 2015, pp. 426–437.
- [14] X. Tan, Y. Zhang, C. Mi, J. Cao, K. Sun, Y. Lin, and M. Yang, Locating the Security Patches for Disclosed OSS Vulnerabilities with Vulnerability-Commit Correlation Ranking, in *Proc. ACM SIGSAC Conf. on Computer and Communications Security*, New York, NY, USA, 2021, pp. 3282–3299.
- [15] J. Devlin, M. W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, *arXiv preprint arXiv: 1810.04805*, 2018.
- [16] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, et al. Language models are few-shot learners, *arXiv preprint arXiv: 2005.14165*, 2020.
- [17] F. Li and V. Paxson, A Large-Scale Empirical Study of Security Patches, in *Proc. ACM SIGSAC Conf. on Computer and Communications Security*, New York, NY, USA, 2017, pp. 2201–2215.
- [18] X. Wang, K. Sun, A. Batcheller and S. Jajodia, Detecting “0-Day” Vulnerability: An Empirical Study of Secret Security Patch in OSS, in *Proc. 49th Annual IEEE/IFIP Int. Conf. on Dependable Systems and Networks*, Portland, OR, USA, 2019, pp. 485–492.
- [19] X. Wang, S. Wang, P. Feng, K. Sun and S. Jajodia, PatchDB: A Large-Scale Security Patch Dataset, in *Proc. 51st Annual IEEE/IFIP Int. Conf. on Dependable Systems*

- and Networks, Taipei, China, 2021, pp. 149–160.
- [20] S. Wang, Y. Zhang, L. Bao, X. Xia and M. Wu, VCMatch: A Ranking-based Approach for Automatic Security Patches Localization for OSS Vulnerabilities, in *Proc. IEEE Int. Conf. on Software Analysis, Evolution and Reengineering*, Honolulu, HI, USA, 2022, pp. 589–600.
- [21] K. Shen, Y. Zhang, L. Bao, Z. Wan, Z. Li and M. Wu, Patchmatch: A Tool for Locating Patches of Open Source Project Vulnerabilities, in *Proc. IEEE/ACM 45th Int. Conf. on Software Engineering: Companion Proceedings*, Melbourne, Australia, 2023, pp. 175–179.
- [22] T. Ji, J. Pan, L. Chen and X. Mao, Identifying Supplementary Bug-fix Commits, in *Proc. IEEE 42nd Annual Computer Software and Applications Conf.*, Tokyo, Japan, 2018, pp. 184–193.
- [23] X. Wang, S. Wang, K. Sun, A. Batcheller and S. Jajodia, A Machine Learning Approach to Classify Security Patches into Vulnerability Types, in *Proc. IEEE Conf. on Communications and Network Security*, Avignon, France, 2020, pp. 1–9.
- [24] Z. Fei and J. Rhee, Vulnerability discovery based on source code patch commit mining: a systematic literature review, *Int. Journal of Information Security*, vol. 23, no. 2, pp. 1513–1526, 2024.
- [25] E. Chen, R. Huang, H. S. Chen, Y. H. Tseng and L. Y. Li, GPTutor: a ChatGPT-powered programming tool for code explanation. in *Proc. Int. Conf. on Artificial Intelligence in Education*, Tokyo, Japan, 2023, pp. 321–327.
- [26] T. Joachims, A probabilistic analysis of the Rocchio algorithm with TFIDF for text categorization, in *Proc. Int. Conf. on Machine Learning*, Nashville, TN, USA, 1997, pp. 143–151.
- [27] J. Ramos, Using tf-idf to determine word relevance in document queries, *Proc. of the first instructional conference on machine learning*, vol. 242, no. 1, pp. 29–48, 2003.

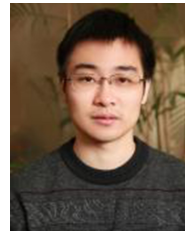


vulnerability patch.

Zhiyu Zhang received the bachelor degree in computer science and technology from Tsinghua University, Beijing, China, in 2021. He is currently pursuing the master degree at Institute for Network Sciences and Cyberspace, Tsinghua University, China. His research interests include natural language processing and



Jianwei Zhuge received the PhD degree from Peking University in 2006, and is an associate research professor in the Institute for Network Science and Cyberspace of Tsinghua University, his research interests include IoT security, software vulnerability detection and mitigation, and supply chain security.



Xiaokang Zhou is currently an associate professor with Faculty of Data Science, Shiga University, Japan. He received the PhD degree in human sciences from Waseda University, Japan, in 2014. He also works as a visiting researcher with RIKEN Center for Advanced Intelligence Project (AIP), RIKEN, Japan, since 2017.

Dr. Zhou has been engaged in interdisciplinary research works in the fields of computer science and engineering, information systems, and social and human informatics. His recent research interests include ubiquitous computing, big data, machine learning, behavior and cognitive informatics, cyber-physical-social systems, and cyber intelligence and security.